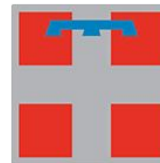




Finanziato
dall'Unione europea
NextGenerationEU



REGIONE
PIEMONTE



DISPENSA DIDATTICA

Corso: B22-484-2025

Programmazione Python



Cofinanziato
dall'Unione europea





JAVASCRIPT

Sviluppare interfacce web interattive



Un po' di storia

- JavaScript è stato inizialmente sviluppato dalla Netscape Communications (1995) con il nome di LiveScript e incluso in Netscape Navigator.
- Il nome è stato poi cambiato in JavaScript in assonanza con il linguaggio Java, con cui in realtà condivide ben poco.
- Successivamente Microsoft ha iniziato a supportare una propria versione JavaScript (detta JScript) in Internet Explorer.
- La vita parallela di questi due linguaggi è durata qualche anno, finchè non si è arrivati allo standard ECMAScript
- Il risultato finale è che un linguaggio standard esiste, ma alcune differenze tra le implementazioni dei vari browser non sono ancora state eliminate...



Cos'è Javascript ?

Javascript è un linguaggio di programmazione di tipo script orientato agli oggetti e agli eventi :

- Non è compilato.
- E' ad alto livello.
- Incorporato o allegato a un HTML, viene interpretato dal browser.
- Rende dinamiche le pagine web.
- Javascript è un linguaggio lato Client. Questo porta numerosi vantaggi: Responsività, Minore carico del lavoro per il server.



Rendere interattive le pagine web

- Accesso ai contenuti
- Modifica dei contenuti: si possono aggiungere o rimuovere contenuti di una pagina HTML; modificare i valori degli attributi “class” per attivare nuove regole CSS
- Programmare regole: implementare procedure e controlli
- Reazione agli eventi: talvolta uno script deve essere eseguito quando si verifica un determinato evento.



HTML & CSS

< html >

Livello dei contenuti

{ Css }

Livello della
presentazione

Javascript()

Livello dei
comportamenti

<p class = "frutto"> pesca </p>

nome
attributo

valore
attributo



HTML & CSS

```
.frutto { color: pink; }
```

nome
proprietà

valore
proprietà



Collegare un file JAVASCRIPT ad una pagina HTML

Creare un elemento `<script>` per chiedere al browser di caricare lo script.

Solitamente va posizionato prima del tag di chiusura `</body>`

```
<script type="text/javascript" src="js/mioJs.js"></script>
```

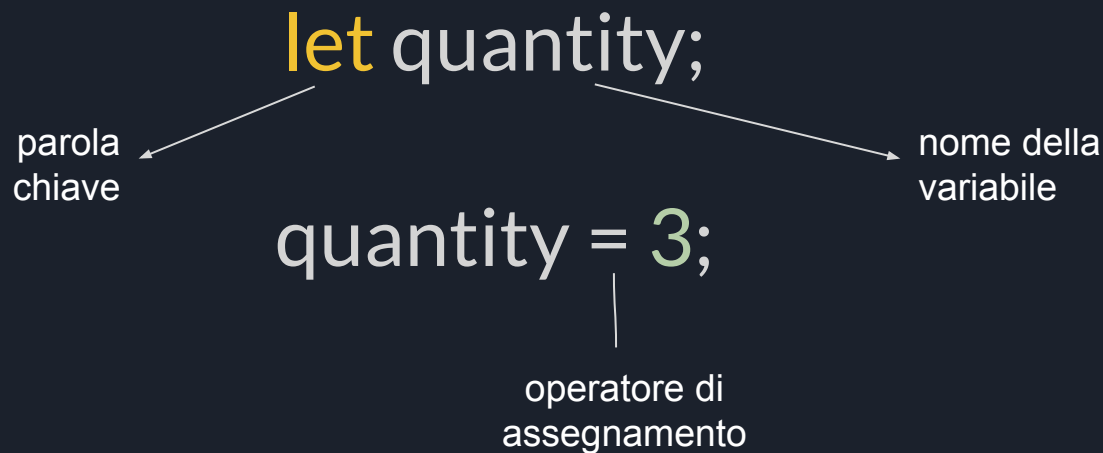
oppure

```
<script> document.write ('<h3>Benvenuto!</h3>')</script>
```




Cos'è una VARIABILE

Compito delle variabili è conservare temporaneamente delle informazioni che servono allo script per eseguire il proprio lavoro. Il nome “variabile” è perfetto poichè il valore può variare ad ogni esecuzione del codice.





Tipi di dati

Numerico

0.75

Stringa

“Ciao!”

Booleano

true/false

Regole denominazione delle variabili

- cominciare con una lettera, _, \$. No numeri
- No - o . all'interno dei nomi della variabile
- OK camelCase
- Descrivere contenuto
- No parole chiave (abstract, boolean, byte, true, try, let, long, ecc ecc)



ARRAY

Per memorizzare un intero elenco di valori correlati. Utili quando non si sa quanti elementi conterrà, per cui non è necessario specificare quanti elementi avrà l'array.

Dichiarazione standard

```
var colors;  
colors = ['bianco', 'rosso', 'blu'];
```

Dichiarazione con costruttore

```
var colors = new Array('bianco', 'rosso', 'blu');
```



Valori per ARRAY

Gli array sono oggetti 0-based , ad ogni elemento viene associato un indice.

```
var itemTre;  
itemTre = colors[2];
```

Per conoscere la dimensione, quindi il numero degli elementi, di un array si utilizza

```
var numColors = colors.length;
```



Metodi per ARRAY

`push()` → aggiunge elementi all'array

`pop()` → rimuove elementi

`concat()` → crea un nuovo array unendone due

`sort()` → ordina gli elementi dell'array

`reverse()` → inverte l'ordine

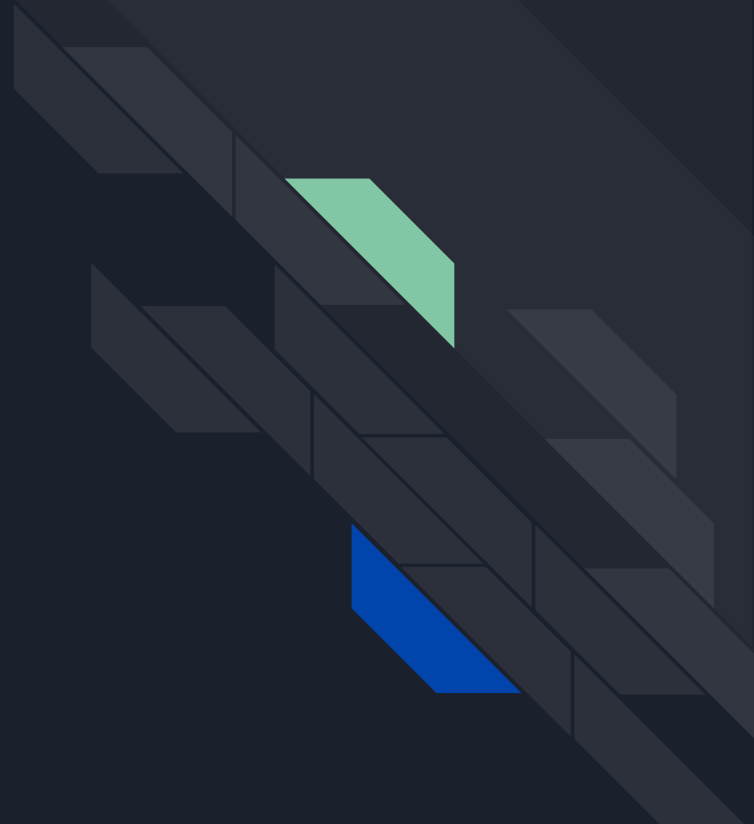


Operatori

Le espressioni contano su elementi chiamati operatori che permettono ai programmatori di creare un unico valore a partire da uno o più valori.

- Operatori di assegnamento `=`
- Operatori aritmetici `+ * - / ++ -- %`
- Operatori per stringhe `+`
- Operatori di confronto `< > <= >=`
- Operatori logici `&& || !`

FUNZIONI, METODI E OGGETTI





Che cos'è una FUNZIONE

Le funzioni consentono di raggruppare tutte le istruzioni necessarie per svolgere un determinato compito.

Le istruzioni di una funzione non vengono eseguite a ogni caricamento della pagina ma ogni qualvolta vengono richiamate.

Alcune funzioni devono ricevere delle informazioni per svolgere le istruzioni, queste informazioni passate alla funzione sono chiamati parametri.

Quando ci si aspetta che una funzione restituisca una risposta, questa è detta valore restituito



Dichiarare e Richiamare una Funzione

Dichiarare

```
function sayHello(){  
    alert('Ciao Cari');  
}
```

Richiamare

```
sayHello();
```



Dichiarare e Richiamare una Funzione con Parametri

Dichiarare

```
function getArea(altezza, larghezza){  
    return altezza * larghezza;  
}
```

Richiamare

```
getArea( 5, 4);
```



Espressioni funzionali

Dichiarare

```
var area = function getArea(altezza, larghezza){  
    return altezza * larghezza;  
}
```

Richiamare

```
var size = area(5,4);
```

Attenzione: un'espressione funzionale non può essere richiamata prima che l'interprete l'abbia raggiunta



Visibilità (SCOPE) delle variabili funzionali

Il punto in cui si dichiara una variabile influenza la posizione in cui può essere utilizzata nel codice.

Scope : campo di visibilità

VARIABILI LOCALI

- livello funzione
- minore utilizzo della memoria

VARIABILI GLOBALI

- livello globale
- maggiore utilizzo della memoria



Richiamare una funzione dal HTML

in HTML

```
<input type = "button" value = "clicca qui" onclick = "Saluta()"
```

in JS

```
Saluta () { alert ('Hello World') ;}
```



OGGETTI

Un oggetto è un modello di entità presente nel mondo reale. In un oggetto variabili e funzioni assumono nuovi nomi

variabile -> proprietà

funzione -> metodo



CREARE UN OGGETTO (notazione letterale)

```
var Studente = {  
  nome: 'Pippo',  
  cognome: 'Ippo',  
  voto: 30,  
  
  superaEsame: function() {  
    alert('Esame superato !!');  
  }  
}
```



ACCEDERE ALLE PROPRIETA'

Notazione a Punto

```
var StudenteNome = Studente.nome;  
var StudenteVoto = Studente.voto;  
var EsameSup = Studente.superaEsame();
```

Notazione parentesi quadre

```
var StudenteNome = Studente['nome'];  
var StudenteVoto = Studente['voto'];  
var EsameSup = Studente['superaEsame']();
```




CREARE UN OGGETTO (notazione a costruttore)

```
var Studente = new Object();
```

```
Studente.nome = 'Pippo';
```

```
Studente.superaEsame = function(){...}
```

Questa notazione è molto utile nel caso in cui si utilizzano più oggetti per rappresentare elementi simili tra loro.



CREARE PIU' OGGETTI (notazione a costruttore)

```
function Studente(nome, cognome, voto){  
    this.nome = nome;  
    this.cognome = cognome;  
    this.voto = voto;  
  
    this.superaEsame = function(){  
        alert('Esame superato !!');  
    }  
}
```

—————> Funzione Costruttore

In questo caso quindi utilizzo una funzione come modello per creare gli oggetti



CREARE PIU' OGGETTI (notazione a costruttore)

```
var Studente1 = new Studente("Mario", "Rossi", 30);
```

```
var Studente2 = new Studente("Paolo", "Bianchi", 28);
```

La funzione costruttore crea istanze dell'oggetto. La parola chiave new seguita dalla chiamata alla funzione crea un nuovo oggetto.



LA PAROLA CHIAVE THIS

Comunemente usate in funzioni e oggetti. La parola chiave this fa sempre riferimento a un oggetto, normalmente quello in cui opera la funzione.

- Quando this viene utilizzato all'interno di una funzione nel contesto globale fa riferimento all'oggetto window.
- Quando this viene utilizzato all'interno di un oggetto e quindi all'interno di un metodo, il this fa riferimento all'oggetto contenitore



OGGETTI BUILT-IN

Gli oggetti built-in rappresentano elementi come la finestra del browser e la pagina web presentata nella finestra. Questi oggetti hanno a disposizione una vasta gamma di metodi e informazioni a cui è possibile accedere

I TRE COMPARTIMENTI PRINCIPALI

BOM

Browser Object Model

DOM

Document Object Model

OGGETTI GLOBALI

String

Number

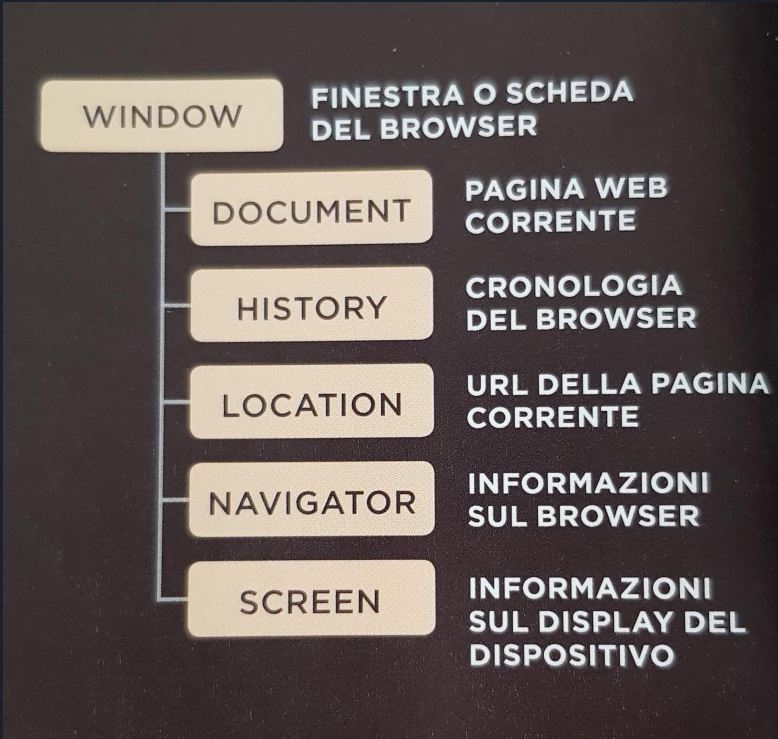
Boolean

Date

Math

BOM (Browser Object Model)

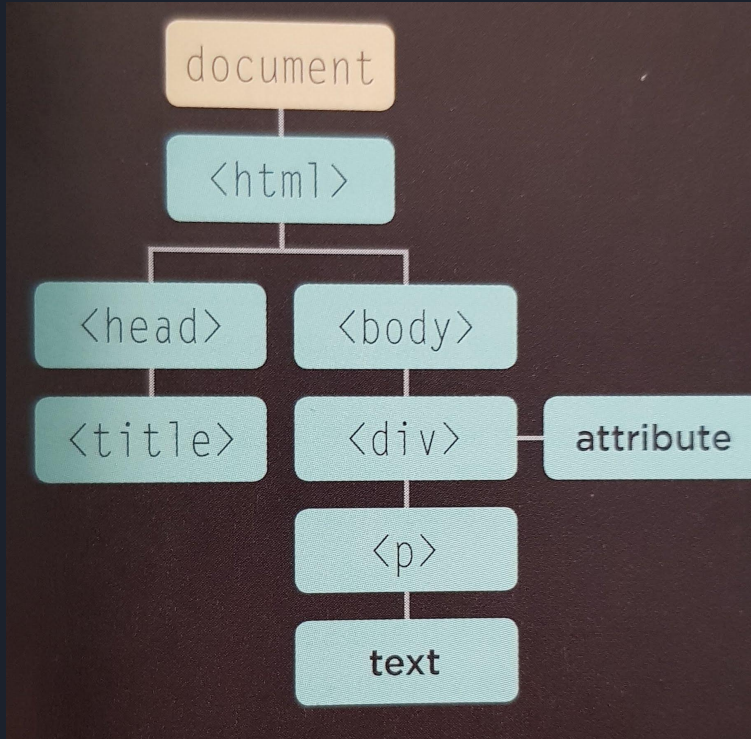
Questo modello contiene oggetti che rappresentano la finestra o la scheda del browser



PROPRIETÀ
<code>window.innerHeight</code>
<code>window.innerWidth</code>
<code>window.pageXOffset</code>
<code>window.pageYOffset</code>
<code>window.screenX</code>
<code>window.screenY</code>
<code>window.location</code>
<code>window.document</code>
<code>window.history</code>
<code>window.history.length</code>
<code>window.screen</code>
<code>window.screen.width</code>
<code>window.screen.height</code>
METODO
<code>window.alert()</code>
<code>window.open()</code>
<code>window.print()</code>

DOM (Document Object Model)

Il DOM crea un modello della pagina web corrente utilizzando gli oggetti. L'oggetto più elevato della gerarchia è document



PROPRIETÀ

- `document.title`
- `document.lastModified`
- `document.URL`
- `document.domain`

Il DOM è fondamentale per accedere e per modificare il contenuto della pagina web corrente.

METODO

- `document.write()`
- `document.getElementById()`
- `document.querySelectorAll()`
- `document.createElement()`
- `document.createTextNode()`



Oggetti Globali: STRING

A una stringa di testo si possono applicare le proprietà e i metodi dell'oggetto String

Proprietà:

length

Metodi:

toUpperCase()

toLowerCase()

charAt()

indexOf()

lastIndexOf()

substring()

split()

trim()

replace()



Oggetti Globali: NUMBER

Quando si usa un valore numerico, si possono applicare le proprietà e i metodi dell'oggetto Number

Metodo:

isNaN()

toFixed()

toPrecision()

toExponential()



Oggetti Globali: MATH

L'oggetto Math offre proprietà e metodi per le costanti e le funzioni matematiche

Metodo:

`Math.round()`

`Math.sqrt(n)`

`Math.ceil()`

`Math.floor()`

`Math.random()`



DECISIONI E CICLI

DECISIONI E CICLI

In un diagramma di flusso il codice può seguire più percorsi. Controllare il flusso dei dati per gestire situazioni diverse è fondamentale.



Per prendere una decisione il codice controlla lo stato attuale di uno script confrontando due valori, tramite un operatore di confronto.



Operatori di Confronto

<code>==</code>	→	è uguale a
<code>===</code>	→	è strettamente uguale a (tipo)
<code>!=</code>	→	non è uguale a
<code>!==</code>	→	non è strettamente uguale a
<code>></code>	→	maggiore di
<code><</code>	→	minore di
<code>>=</code>	→	maggiore uguale
<code><=</code>	→	minore uguale



Operatori logici

`&&` $\longrightarrow$ AND logico

`||` $\longrightarrow$ OR logico

`!` $\longrightarrow$ NOT logico

tabella true/false



Usare le istruzioni IF

L'istruzione if valuta una condizione. Se questa condizione è true allora viene eseguito il blocco di codice successivo

```
if (ore > 10 ){  
    fai qualcosa..  
}
```



IF ... ELSE

L'istruzione if valuta una condizione. Se questa condizione è true allora viene eseguito il blocco di codice successivo, altrimenti viene eseguito il blocco di codice successivo all' Else

```
if (ore > 10 ){  
    fai qualcosa..  
}else{  
    fai altro...  
}
```




SWITCH

L'istruzione switch inizia con una variabile di controllo, seguita dai vari casi.

```
switch(condizione){  
    caso 1:  
        faiqualcosa;  
    break;  
    caso 2:  
        faialtro;  
    break;  
    caso 1:  
        faialtroancora  
    break;  
}
```



CICLI

Anche i cicli si basano su una condizione. Se questa condizione è verificata allora viene eseguito il blocco di codice compreso tra le parentesi graffe. Successivamente la condizione viene ricontrollata e se è ancora **true** viene ripetuto il codice, questo finché la condizione non diventa **false**.

FOR

Esegue un blocco di codice per un numero determinato di volte

WHILE

Il codice continuerà ad essere eseguito finché la condizione resterà **true**

DO...WHILE

simile al while il codice nel do viene eseguito almeno una volta, successivamente viene verificata la condizione



CICLO FOR

```
for (var i = 0; i < 10; i++) {  
    document.write(i);  
}
```

var i = 0 → Inizializzazione

i < 10 → Condizione

i++ → Aggiornamento



FOR ... IN e FOR ... OF

Esistono altri tipi di for per poter scorrere velocemente proprietà di oggetti o Array

For..in

permette di iterare
sulle proprietà
enumerabili degli
oggetti

For..of

permette di ciclare
sugli iterable objects
come String, Array,
List.



WHILE

```
while (condizione){  
    fai qualcosa...  
}
```

DO...WHILE

```
do{  
    fai qualcosa....  
}while(condizione);
```



DOM



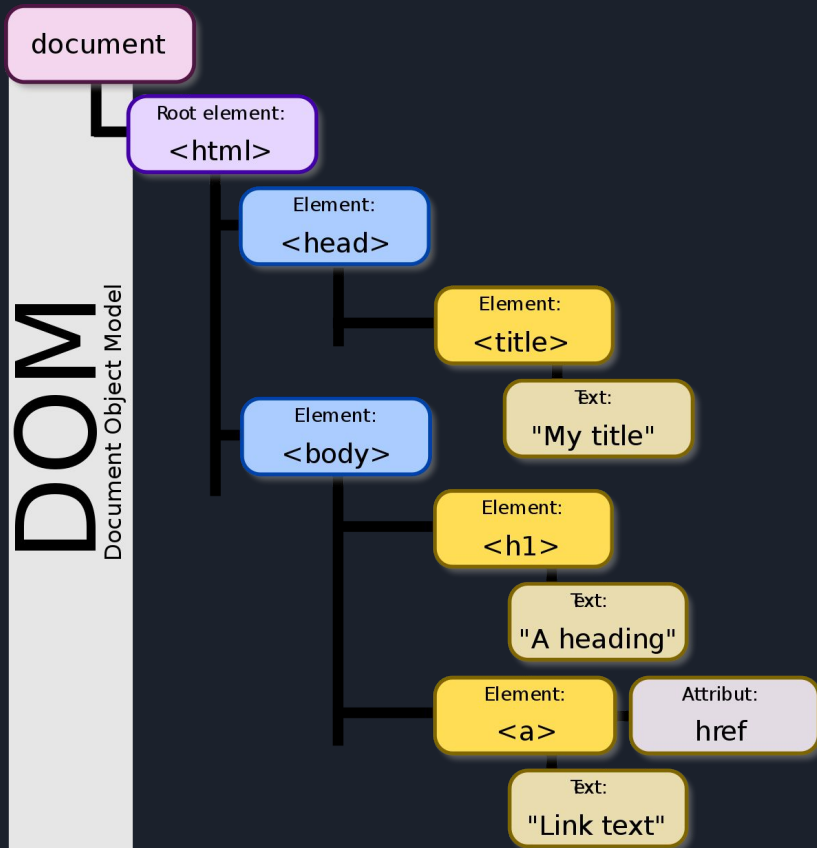
DOM

Il DOM (Document Object Model) modella il modo in cui i browser rappresentano una pagina HTML e il modo in cui Javascript può accedere al contenuto di una pagina web e aggiornarlo mentre è visualizzato in una finestra del browser.

Il Dom è un insieme di regole distinto, implementato dai principali browser, si occupa sostanzialmente di due cose:

- Creare un modello della pagina HTML utilizzando un albero del DOM
- Accedere al contenuto della pagina HTML utilizzando metodi e proprietà degli oggetti del modello. Il DOM è un API (Application Programming Interface)

ALBERO DEL DOM



Quando un browser carica una pagina web crea un modello. Il modello è l'albero del DOM.

Costituito da 4 tipi di nodi:

- nodo del DOCUMENTO (l'oggetto document)
- nodi degli ELEMENTI (<h1>; <a>; <body>)
- nodi degli ATTRIBUTI (class; href)
- nodi di TESTO (questi non possono avere nodi figli nella gerarchia)



OPERARE SULL' ALBERO DEL DOM (1)

1. Accedere agli elementi
1. Operare sugli elementi

METODI PER SELEZIONARE UN SINGOLO ELEMENTO:

- `getElementById('id');`
seleziona un singolo elemento dato il suo valore dell'attributo id
- `querySelector('cssSelector')`
la selezione avviene tramite sintassi css (es 'li.cool'). Restituisce solo il primo elemento del node



OPERARE SULL' ALBERO DEL DOM (2)

METODI PER SELEZIONARE PIU' ELEMENTI :

- `getElementByClassName('class');`
seleziona tutti gli elementi sulla base dell'attributo class
- `getElementsByTagName('tagName');`
seleziona tutti gli elementi che hanno il tag specifico
- `querySelectorAll('cssSelector');`
la selezione avviene tramite sintassi css (es 'li.cool'). Restituisce solo il primo elemento del node

Oss: quando utilizzo questi metodi javascript mi restituisce una NodeList (anche quando trova un singolo elemento)



OPERARE SULL' ALBERO DEL DOM (2)

NODELIST

Le nodeList somigliano agli Array; sono un tipo di oggetto chiamato colletion. È possibile quindi applicare proprietà e metodi tipici degli array:

- `.length( )`
numero degli elementi presenti in una nodeList
- `.item( )`
restituisce un determinato nodo

Oss: nel metodo `item()` va specificato come parametro l'indice dell'elemento desiderato. es. `.item(0)`

Questo metodo può essere avviato con l'uso della sintassi ad Array



TRAVERSING DOM

Dal nodo di un elemento è possibile selezionare altri elementi correlati tramite 5 proprietà:

`.parentNode`

`<li> -> <ul>`

`.previousSibling`

`.nextSibling`

`<li> -> <li>`

`.firstChild`

`.lastChild`

`<ul> -> <li>`

Il traversing può essere reso difficile dal fatto che alcuni browser aggiungono nodi di testo quando ci sono spazi vuoti.



ACCEDERE / AGGIORNARE IL CONTENUTO DI UN ELEMENTO

Quando si lavora con il nodo di un elemento (e non col suo nodo di testo), questo può contenere del codice. Per cui la scelta della tecnica da utilizzare dipende dal contenuto dell'elemento.

`.innerHTML`

legge/scrive testo e codice

`.textContent`

legge/scrive solo testo

`.innerText`

legge/scrive solo testo

`.innerHTML` ottiene il contenuto di un elemento e lo restituisce come stringa

`.innerText` non consigliato il suo utilizzo per problemi di: supporto browser; CSS; prestazioni.



AGGIUNGERE ELEMENTI TRAMITE MANIPOLAZIONE DEL DOM

Manipolare il Dom rappresenta un'altra tecnica per aggiungere nuovi contenuti ad una pagina (da utilizzare al posto di innerHTML)

`createElement( )`

crea l'elemento

`createTextNode( )`

crea un nuovo nodo di
testo

`appendChild( )`

specifica a quale
elemento deve essere
aggiunto sotto forma di
figlio

`removeChild( )`

rimuove l'elemento figlio
dopo aver salvato il
contenitore genitore in
una variabile



CONTROLLARE ATTRIBUTO E OTTENERE VALORI

Serie di metodi per il controllo e la gestione degli attributi dei tag HTML

```
.hasAttribute('nomeAttributo')
```

```
.getAttribute('class')
```

```
.setAttribute('class', 'blue')
```

```
.removeAttribute('class')
```



EVENTI



EVENTI

Il browser registra tutti i tipi di eventi. Lo script risponde a questi eventi aggiornando il contenuto della pagina, rendendola più interattiva:

- le interazioni creano gli eventi (click o tap su link) ;
- gli eventi lanciando codice ;
- il codice risponde agli utenti tramite interventi sul DOM;

SELEZIONE DI EVENTI

- eventi UI (load, unload, error, resize) ;
- eventi tastiera (keydown, keyup, keypress);
- eventi Mouse (click, dbclick, mouseover, scrollup);
- eventi Focus (focus / focusin , blur / focusout);
- eventi per moduli (input, change, submit, reset, copy, cut, paste, select);



GESTIONE DEGLI EVENTI

- 1) Selezionare il nodo dell'elemento cui lo script deve rispondere;
- 2) indicare quale evento lancerà la risposta (binding);
- 3) indicare il codice da eseguire;

TRE MODI PER COLLEGARE UN EVENTO

GESTORI EVENTI
HTML

`<onclick = "hide()">`

GESTORE EVENTI
DOM

Supporto browser elevato. Può essere collegata un'unica funzione.

LISTENER DI EVENTI

Metodo di gestione moderno e da preferire per il suo utilizzo.



GESTORE TRADIZIONALE

`elemento.onevento = nomeFunzione;`



Le parentesi vengono omesse
perché non si vuole che la
funzione si verifichi prima di
richiamare l'evento

es:

`div.onclick = rimuovi;`



LISTENER DI EVENTI

`elemento.addEventListener ('evento', 'nomeFunzione, Boolean);`



Flusso dell'evento, legato alla risalita e al richiamo di un evento.

Solitamente viene settato a `false`